

# Why MCP Needs Runtime Evidence: The Security Layer Between AI Agents and Real-World Tools

---

By Mats Sundvall · July 14, 2026 · [github.com/matssun/mcp-re](https://github.com/matssun/mcp-re)

AI systems are moving from answering questions to taking action.

That change is much bigger than it first appears.

A chatbot that answers a question is mostly producing text. An agent that calls tools is doing something else entirely. It may read email, query a database, create a GitHub issue, update a calendar, open a ticket, call an internal API, fetch patient data, deploy code, or trigger a business process.

Once that happens, the tool call is no longer just a technical message.

It is a security event.

And that leads to a simple but uncomfortable question:

When a tool receives a request from an AI agent, what does it actually know?

Does it know who asked?

Does it know whether the request was changed?

Does it know whether the request was replayed?

Does it know whether the request was intended for this tool?

Does the client know that the response really belongs to the request?

Can anyone prove what happened afterwards?

This is the problem runtime evidence is meant to solve.

## MCP makes agents useful

The Model Context Protocol, MCP, gives AI systems a standard way to talk to tools.

That is powerful. Instead of every tool vendor inventing a separate integration model, MCP gives clients and servers a shared protocol for exposing capabilities to AI systems. This makes agentic workflows much more practical.

But usefulness changes the risk profile.

A tool that only returns public information is one thing. A tool that can access private documents, internal systems, production infrastructure, financial data, medical records, or customer information is something else.

The moment an AI agent can use a tool to perform real work, the tool call becomes part of the security boundary.

That boundary needs more than hope, logs, and a valid-looking network connection.

## TLS and API tokens are not enough

Most people's first reaction is understandable:

"We already have TLS."

"We already use API tokens."

"The server has logs."

"The user approved the app."

All of those may be useful. None of them is enough by itself.

TLS protects the connection between two endpoints. It does not, by itself, give every downstream participant cryptographic evidence of the full runtime meaning of a tool call.

An API token may prove that something had access. It does not necessarily prove exactly what request was made, what bytes were bound to the request, whether the request was fresh, whether it was replayed, or whether a response belongs to that specific request.

Logs are important, but logs are usually assertions made by systems after the fact. They are not the same thing as portable, verifiable evidence.

This is the gap.

Modern agent systems do not just need access control. They need runtime evidence.

## What runtime evidence means

Runtime evidence means that a request or response carries enough cryptographic proof that another party can verify the important facts.

For an MCP tool call, those facts include things like:

- who signed the request;
- what exact bytes were signed;
- which audience the request was intended for;
- whether the request was fresh;
- whether the request was replayed;
- whether the response is bound to the request;
- whether a rejection or error was produced by the real security boundary;
- whether a delegated signing key was valid at the time.

The point is not merely to log that something happened.

The point is to make the important parts verifiable.

MCP-RE is built around this idea: MCP Runtime Evidence. It does not ask every participant to simply trust that a tool call happened correctly. It gives them evidence they can verify.

## What can go wrong without evidence

Without runtime evidence, many failures become hard to distinguish from normal operation.

A valid-looking request may be replayed.

A response may be accepted without proof that it belongs to the request that caused it.

A tool may receive a request that was intended for another audience.

A client may receive an unsigned error and have no reliable way to know whether it came from the real security boundary or from a broken path.

An agent may act on behalf of a user, but the receiving tool may not be able to verify the delegation chain.

An incident may occur, and the logs may show what systems claim happened, but not provide cryptographic evidence of what was signed, accepted, rejected, and returned.

That is not a theoretical concern. It is exactly the kind of ambiguity that becomes dangerous when AI agents are allowed to act across real systems.

If an agent can trigger meaningful work, then the runtime path around that work needs to be verifiable.

## What MCP-RE verifies

MCP-RE is not a magic security layer that solves every possible MCP problem.

It does not fix a badly written tool. It does not decide whether a business rule is correct. It does not eliminate prompt injection. It does not replace identity providers, policy engines, secret management, tenant isolation, data governance, or secure software engineering.

Its job is narrower and more precise.

MCP-RE focuses on making MCP runtime interactions verifiable at the evidence boundary.

That includes:

- signing requests;
- binding the message body;
- binding the intended audience;
- checking freshness;
- blocking replay;
- verifying responses;
- producing verifiable rejection evidence;
- supporting delegated signing so production systems do not need root keys on the hot path.

That boundary matters because an AI tool call is not just an API call. It is often an action performed in a wider chain of agency, delegation, policy, and audit.

MCP-RE gives that action evidence.

## Zero Trust-aligned, not magic security

It is tempting to say that MCP-RE “makes MCP Zero Trust.” That would be too broad.

Zero Trust does not mean that everything is automatically secure. It means that important decisions should be explicitly verified rather than implicitly trusted.

MCP-RE is better described as a Zero Trust-aligned runtime evidence layer for MCP.

It helps answer questions such as:

- was this request signed by the expected actor?
- was this request intended for this audience?
- has this request already been seen?
- does this response belong to this request?
- was this delegated signing key valid when it was used?

Those are Zero Trust-style questions. They are about verification, not assumption.

But MCP-RE does not secure everything above or below that boundary. The application still needs authorization logic. The tool still needs to be safe. The model still needs guardrails. The infrastructure still needs hardening.

Runtime evidence is one missing layer, not the whole security universe.

## Why standards alignment matters

Early designs for runtime evidence can easily drift toward custom cryptography and custom message formats. That is dangerous.

Security profiles should not invent new protocols when existing standards fit the problem.

The direction of MCP-RE has therefore moved toward aligning MCP runtime evidence with existing web and security standards.

The key standards are:

- HTTP Message Signatures (RFC 9421), for signing selected HTTP request and response components;
- Content-Digest / Digest Fields (RFC 9530), for binding the message body;
- JOSE/JWS credentials (RFC 7515, with RFC 7800 proof-of-possession and RFC 8037 Ed25519), for delegated signing evidence;
- OAuth-style sender-constrained patterns, such as DPoP-style key binding, where applicable.

The goal is not to replace the web security stack.

The goal is to make MCP runtime evidence fit into it.

This matters for adoption. Enterprises already have gateways, key-management systems, audit systems, cloud KMS products, identity providers, and operational security practices. A credible MCP security profile must compose with that world instead of asking everyone to trust an entirely new island.

## Why the HTTP profile matters

MCP can appear in different transport contexts, but production security boundaries need clarity.

The current direction is HTTP-profile focused: HTTP in, HTTP out.

That is important because HTTP is where the relevant security and operational standards already exist. It is also where production infrastructure is strongest: load balancing, observability, routing, mTLS, KMS integration, service identity, traffic policy, and fleet operation.

This does not mean that every MCP client or server in the ecosystem already speaks HTTP directly.

If a client or server only speaks stdio, that can be handled by a plain MCP transport adapter outside the evidence layer. The evidence boundary itself should not become responsible for every possible transport conversion.

That separation keeps the design clean:

- adapters translate transports;
- MCP-RE provides runtime evidence;
- the application provides domain behavior.

Mixing those responsibilities would make the security boundary harder to reason about.

## The production problem: evidence must scale

A runtime evidence layer is not useful if it only works in a small local demonstration.

If MCP becomes a serious enterprise integration layer, evidence has to work under real production conditions.

That means concurrent users, many agents, many tool calls, rolling updates, replica failures, key rotation, replay attempts, and real network infrastructure.

A tool call itself may take seconds. It may search documents, query a CRM, wait for a database, call a third-party API, or trigger a long-running workflow.

But the evidence decision around that call should not take seconds.

The security layer must verify, reject, sign, and route predictably. It must not become the bottleneck. It must not lose replay coherence when the system is scaled across replicas. It must not depend on a root key being used on every hot-path response.

For enterprise use, this kind of layer has to be architected for thousands of transactions per second, even when the underlying tool calls themselves may be slower.

That requirement changes the implementation.

## Evidence has to work under real load

MCP-RE is implemented in Rust because the evidence boundary sits on the hot path.

The job of that boundary is not simply to parse messages. It must perform security-critical work with predictable latency: verify signatures, bind requests to audiences, enforce freshness, block replay, sign responses or rejections, and route traffic without collapsing under concurrency.

That is why implementation architecture matters.

The current implementation has been validated on a live Google Kubernetes Engine fleet, not only in local tests. On real GKE hardware, MCP-RE has proven:

- **cross-replica replay coherence** — a nonce accepted by one replica is rejected as a replay by a sibling, because the freshness state lives in a shared tier, not in one process;
- **cross-replica trust revocation** — advancing the shared trust epoch invalidates a previously valid credential across every replica at once;
- **multi-round-trip continuation across a replica switch** — a stateful interaction opened on one replica is honoured on another;
- **delegated authority rooted in Cloud KMS via Workload Identity** — the enterprise custody model, described below, running with the root key reached only through the platform's own identity, never as key material inside the pod.

And it is measured, not asserted.

The first published baseline is deliberately conservative: cold TLS 1.3 mutual-TLS, 128-way concurrency, 8,000 requests per run, and every request accounted for. It is a regression envelope for the full security path, not the expected steady-state keep-alive or HTTP/2 production ceiling.

| replica hardware | 8-core verified throughput | per-core scaling |
|------------------|----------------------------|------------------|
| e2-standard-8    | ~396 responses/sec         | ~0.70 of linear  |
| c3-standard-8    | ~493 responses/sec         | ~0.67 of linear  |

Those are cold-connection envelope figures for regression detection, not marketing numbers. Production deployments can reuse connections and are expected to behave differently under steady-state traffic. The important point is that the baseline is measured on real cloud hardware, under load, with the full evidence path active on every request.

Running on real infrastructure is not ceremony. The live Workload Identity tests exercise failure modes that local and kind-based tests do not reach: cloud metadata, service identity, token acquisition, KMS access, and pod-level runtime behavior. That is exactly why the system is tested on a real GKE fleet rather than only in local emulation.

That matters because evidence cannot only be correct in a unit test.

It has to remain correct while the fleet is under load.

It has to remain correct when two replicas see the same request.

It has to remain correct during rolling updates.

It has to remain correct when keys are rotated.

It has to remain correct when the system is attacked with replays.

A security profile that fails under concurrency is not a security profile. It is a demo.

## Keeping root keys off the hot path

Delegated signing is a good example of why implementation detail matters.

A naive design signs every response with the server's root identity key.

If that key lives in an HSM or a cloud KMS, which for a production identity it should, then every response becomes a remote call into the KMS.

That is a latency tax on every request and a ceiling on throughput.

MCP-RE avoids this.

The root key stays in the KMS and signs only a short-lived delegation credential, and only when a key is issued or rotated.

That credential is a standards-based JOSE/JWS token: RFC 7515, with an RFC 7800 proof-of-possession claim binding an Ed25519 key per RFC 8037.

It authorizes a short-lived, in-memory delegated key to sign responses on the root's behalf.

It travels inline in the same RFC 9421 evidence, so a verifier can check the whole chain back to the root without any out-of-band lookup.

Per-request signing then happens in memory, in microseconds, while the root key is touched only at rotation.

The verifier still gets a full cryptographic chain: this delegated key was authorized by the root, within a bounded lifetime, for this audience.

Rotation overlaps, so there is no signing gap.

Issuance fails closed rather than extending an expired key.

Revocation propagates through the same trust-epoch channel the rest of the system already uses.

And the load-bearing property is verifiable, not just asserted: the per-request path performs zero remote KMS operations.

That is proven in the test suite and exercised against live Google Cloud KMS — including on a live GKE fleet where the root key is reached only through **Workload Identity**, the platform's own service-identity mechanism, so no private key material ever enters the pod and no static credential is deployed. Signing a whole batch of responses calls the KMS only at issuance or rotation, and never on the hot path.

This is the custody model enterprises already expect: the identity root lives in the cloud KMS under existing key-management controls, the workload authenticates with platform identity rather than a copied secret, and the delegation credential — a standards-based JOSE/JWS token — carries the whole chain inline so a verifier needs no out-of-band lookup.

Delegated authority, KMS-rooted, with the root kept off the hot path, is the design point — and it runs on real cloud infrastructure.

## What v0.12 represents

MCP-RE v0.12 is best understood as the standards-alignment release, and v0.12.1 is where that direction was proven on real cloud infrastructure.

It draws a clearer boundary around what runtime evidence for MCP should be:

- standards-aligned HTTP evidence;
- request and response binding;
- replay coherence, held across replicas;
- delegated signing custody, rooted in Cloud KMS via Workload Identity;
- HTTP-focused production operation, validated on a live GKE fleet;
- removal of transport confusion from the core security story.

This is the combination that makes MCP-RE a candidate for enterprise use rather than a demo: delegated authority under existing key-management controls, evidence that stays coherent when the system is scaled across replicas, and a measured SLO baseline on real hardware.

The remaining hardening work is around cloud load-balancer handoff during rolling updates: making the already-tested drain behavior hold cleanly through the full managed load-balancer path. That work is tracked openly, because runtime evidence should be validated under the same real conditions it is designed to protect.

The important part is not one feature in isolation.

The important part is that the evidence layer now has a clearer shape:

MCP gives agents a way to act.

MCP-RE gives those actions evidence.

## Who should care

If you are building a toy integration, this may feel like too much.

If you are building systems where AI agents can affect real data, real users, real infrastructure, or real business processes, it is not too much.

The people who should care include:

- MCP server authors;
- AI platform teams;
- enterprise security architects;
- SaaS vendors exposing tools to agents;
- teams building internal agentic workflows;
- infrastructure teams responsible for audit and incident response;
- standards people thinking about how agent systems should be secured.

The simple rule is this:

If an AI agent can cause a real action in your system, you should care about runtime evidence.

## Why evidence becomes non-optional

That may sound dramatic, but the concern is straightforward.

If your MCP tools can perform meaningful work and you cannot prove who requested what, what was signed, whether it was replayed, and whether the response belongs to the request, then you do not have a runtime security story.

You have trust in the happy path.

You have logs.

You have assumptions.

Those may be enough for experiments. They are not enough for high-value agentic systems.

As agents become more capable, the question will not only be whether they are useful. The question will be whether their actions are verifiable.

## From trust to evidence

The future of agentic systems will not be secured by asking everyone to trust every intermediary, every log line, and every runtime assumption.

It will be secured by making the important parts verifiable.

That is the role of runtime evidence.

It does not replace application security. It does not replace authorization policy. It does not replace careful tool design.

But it gives the runtime interaction itself something it badly needs:

proof.

Once agents can act through tools, every tool call becomes a security event.

MCP needs runtime evidence so those events are not merely trusted, logged, or inferred, but cryptographically verifiable.